

ARSENA Claire - TD3, Introduction à l'intelligence artificielle

▼ Exercice 1

1) Preuve d'insatisfiabilité de $S = \{P, \neg P \vee Q, \neg Q \vee R, \neg Q \vee \neg R\}$:

1. P
2. $\neg P \vee Q$
3. $\neg Q \vee R$
4. $\neg Q \vee \neg R$
5. Q : Résolution entre 1 et 2
6. R : Résolution entre 5 et 3
7. $\neg R$: Résolution entre 5 et 4
8. Clause vide : Résolution entre 6 et 7

L'ensemble est bien insatisfiable

2) L'ensemble $S = \{P \vee Q, P \vee \neg Q, R \vee Q, R \vee \neg Q\}$ est-il satisfiable ?

- En résolvant $(P \vee Q)$ et $(P \vee \neg Q)$, on obtient P .
- En résolvant $(R \vee Q)$ et $(R \vee \neg Q)$, on obtient R .

On arrive à saturation sans trouver une clause vide, donc l'ensemble est satisfiable

3) Montrer que R est la conséquence de l'ensemble

Pour prouver que R est vrai, on utilise la **réfutation** : on ajoute $\neg R$ à l'ensemble et on cherche la contradiction ($\perp$)

L'ensemble de départ :

1. $P \vee Q \vee R$
2. $\neg P \vee Q \vee R$
3. $\neg Q \vee R$
4. $\neg R$
5. $P \vee Q$: Résolution entre 1 et 4
6. $\neg P \vee Q$: Résolution entre 2 et 4
7. $\neg Q$: Résolution entre 3 et 4
8. Q : Résolution entre 5 et 6
9. Clause vide : Résolution entre 7 et 8

On a trouvé la contradiction, donc R est bien prouvé

▼ Exercice 2

On transforme les implications en "ou" pour que l'ordinateur puisse calculer

- $C1 : \neg vivant(X) \vee \neg mange(X, viande) \vee carnivore(X)$
- $C2 : animal(X) \vee vegetal(X)$
- $C3 : \neg animal(X) \vee vivant(X)$
- $C4 : \neg vegetal(X) \vee vivant(X)$

On a : $mange(dagobert, viande)$, $animal(dagobert)$

On veut prouver $carnivore(dagobert)$. On commence donc par le but négatif : $\neg carnivore(dagobert)$

1. On utilise l'**unification** : on remplace X par dagobert dans **C1**.
2. On mélange **C1** avec notre but $\neg carnivore(dagobert)$. Il reste :
 $\neg vivant(dagobert) \vee \neg mange(dagobert, viande)$
3. On sait que $mange(dagobert, viande)$ est vrai.
Il ne reste qu'à prouver : $\neg vivant(dagobert)$
4. Avec **C3** et le fait $animal(dagobert)$, on prouve que dagobert est vivant

▼ Exercice 3

Partie 1

On crée le fichier td3.pl dont le contenu pour la question 1 est le suivant :

```
homme(albert).
homme(jean).
homme(paul).
homme(bertrand).
homme(louis).
homme(benoit).

femme(marie).
femme(germaine).
femme(sophie).
femme(christiane).
femme(simone).

pere(louis, benoit).
pere(albert, jean).
pere(jean, simone).
pere(jean, paul).
pere(paul, sophie).
pere(paul, bertrand).

mere(marie, sophie).
mere(marie, bertrand).
mere(germaine, jean).
mere(christiane, simone).
mere(christiane, paul).
mere(simone, benoit).
```

En suite, on lance swipl dans le bon répertoire

```
PS C:\Users\clair\Desktop\IUT-1\2E ANNEE 25-26\S4\R4.A.L
1 - Introduction à l'IA (2)> swipl
```

Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.1)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run `?- license.` for legal details.

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

```
1 ?- [td3] .  
true.  
  
2 ?- homme(paul) .  
true.  
  
3 ?- femme(benoit) .  
false.  
  
4 ?- femme(X) .  
X = marie .  
  
5 ?- homme(X) .  
X = albert .  
  
6 ?- mere(marie, sophie) .  
true.  
  
7 ?- mere(marie, benoit).  
false.  
  
8 ?- mere(X, jean) .  
X = germaine.  
  
9 ?- pere(paul, X) .  
X = sophie  
  
10 ?- homme(X), pere(X, _) .  
X = albert .  
  
11 ?- pere(X, _) .  
X = louis .
```

questions sous forme de requêtes prolog

Partie 2

Contenu à rajouter dans le `td3.pl`

```

parent_de(X, Y) :- pere(X, Y).
parent_de(X, Y) :- mere(X, Y).

fils_de(X, Y) :- homme(X), parent_de(Y, X).

fille_de(X, Y) :- femme(X), parent_de(Y, X).

enfant_de(X, Y) :- parent_de(Y, X).

grand_pere_de(X, Y) :- pere(X, Z), parent_de(Z, Y).

grand_mere_de(X, Y) :- mere(X, Z), parent_de(Z, Y).

frere_de(X, Y) :- homme(X), pere(P, X), pere(P, Y), mere
(M, X), mere(M, Y), X \= Y.

soeur_de(X, Y) :- femme(X), pere(P, X), pere(P, Y), mere
(M, X), mere(M, Y), X \= Y.

demi_frere_de(X, Y) :- homme(X), parent_de(P, X), parent
_de(P, Y), parent_de(AutreP1, X), parent_de(AutreP2, Y),
AutreP1 \= P, AutreP2 \= P, AutreP1 \= AutreP2, X \= Y.

demi_soeur_de(X, Y) :- femme(X), parent_de(P, X), parent
_de(P, Y), parent_de(AutreP1, X), parent_de(AutreP2, Y),
AutreP1 \= P, AutreP2 \= P, AutreP1 \= AutreP2, X \= Y.

```

pour ce cas, il n'y a aucun demi frère ni demi soeur, il faudrait rajouter d'autres règles.

▼ Exercice 4

```

aime(marie, vin).
aime(pierre, X) :- aime(X, vin).
voleur(pierre).
vole(X, Y) :- voleur(X), aime(X, Y).

```

on pose donc la requête et on obtient que Pierre vole le vin

```
?- vole(pierre, Quoi).  
Quoi = vin.
```

▼ Exercice 5

1. Sans le cut, prolog trouve Y = negatif pour -1, mais il continue de chercher si -1 pourrait aussi être zero ou positif. C'est inefficace car un nombre n'a qu'un seul signe.
2. On utilise le cut pour supprimer les branches inutiles une fois la réponse trouvée.

```
sign(X, negatif) :- X < 0, !.  
sign(X, zero) :- X == 0, !.  
sign(X, positif) :- X > 0.
```

▼ Exercice 6 : Les Listes

En Prolog, une liste est un ensemble de Tête | Queue (`[T|Q]`).

1. Éléments de rang impair

On prend le premier élément, on saute le deuxième, et on continue pour le reste

```
elements_impairs_de([], []).  
elements_impairs_de([X], [X]).  
elements_impairs_de([X, _|Reste], [X|Resultat]) :- elements_impairs_de(Reste, Resultat).
```

```
7 ?- elements_impairs([a, b, c, d, e], L).  
L = [a, c, e] .
```

2. Inverser une liste

On détache la tête, on inverse le reste, puis on la recolle à la fin.

```
inverse([], []).  
inverse([T|Q], LInv) :- inverse(Q, QInv), append(QInv,
```

[T], LInv).

```
8 ?- inverse([1, 2, 3], L).  
L = [3, 2, 1].
```

Sujet du TD3